



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CS304 SOFTWARE ENGINEERING

Yida Tao

taoyd@sustech.edu.cn

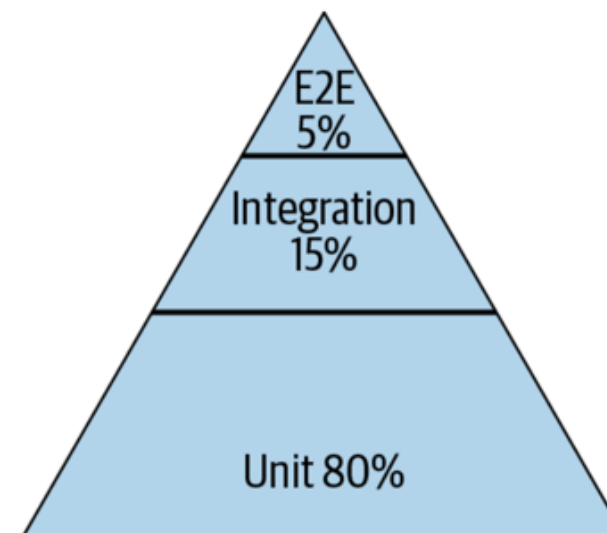


LECTURE 9

- Unit tests: best practices
- Integration tests: Test Doubles
- UI Testing
- Case study: HarmonyOS App Testing



MAINTAINABLE UNIT TESTS





ULTIMATE GOAL: UNCHANGING TESTS

- **Ideally**, after a test is written, it never needs to change **unless the requirements of the system under test change**
- Maintainable tests “just work”: after writing them, engineers don’t need to think about them again until they fail, and those failures indicate **real bugs** with clear causes.

Only changing the system’s requirements would require the updates to existing tests



GOOD PRACTICE 1: TEST VIA PUBLIC APIS

- Write tests that invoke the SUT in the same way its users would
- That is, making calls to public APIs rather than private ones (implementation details)

BAD PRACTICE

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
    if (isValid(transaction)) {
        saveToDatabase(transaction);
    }
}

private boolean isValid(Transaction t) {
    return t.getAmount() < t.getSender().getBalance();
}

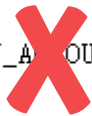
private void saveToDatabase(Transaction t) {
    String s = t.getSender() + ", " + t.getRecipient() + ", " + t.getAmount();
    database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
    // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
    // Read transactions from the database to determine the account balance
}
```

BAD PRACTICE

```
@Test
public void emptyAccountShouldNotBeValid() {
    assertThat(processor.isValid(newTransaction().setSender(EMPTY_ACCOUNT)))
        .isFalse();
}
```



Test the internal (private) implementation logic directly.
This is NOT how real users would use the system

Even minor refactorings that are invisible to users can
cause the test to break

- Renaming `isValid()`

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
    if (isValid(transaction)) {
        saveToDatabase(transaction);
    }
}

private boolean isValid(Transaction t) {
    return t.getAmount() < t.getSender().getBalance();
}

private void saveToDatabase(Transaction t) {
    String s = t.getSender() + ", " + t.getRecipient() + ", " + t.getAmount();
    database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
    // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
    // Read transactions from the database to determine the account balance
}
```

BAD PRACTICE

```
@Test
public void shouldSaveSerializedData() {
    processor.saveToDatabase(newTransaction()
        .setId(123)
        .setSender("me")
        .setRecipient("you")
        .setAmount(100));
    assertThat(database.get(123)).isEqualTo("me,you,100");
}
```



Test the internal states of the database.

Even minor refactorings that are invisible to users can cause the test to break

- Changing how String s is constructed in saveToDatabase()

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
    if (isValid(transaction)) {
        saveToDatabase(transaction);
    }
}

private boolean isValid(Transaction t) {
    return t.getAmount() < t.getSender().getBalance();
}

private void saveToDatabase(Transaction t) {
    String s = t.getSender() + "," + t.getRecipient() + "," + t.getAmount();
    database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
    // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
    // Read transactions from the database to determine the account balance
}
```


TEST VIA PUBLIC APIS

```
@Test
public void shouldTransferFunds() {
    processor.setAccountBalance("me", 150);
    processor.setAccountBalance("you", 20);

    processor.processTransaction(newTransaction()
        .setSender("me")
        .setRecipient("you")
        .setAmount(100));

    assertThat(processor.getAccountBalance("me")).isEqualTo(50);
    assertThat(processor.getAccountBalance("you")).isEqualTo(120);
}
```



The test access the system in the same manner as the users would via public APIs

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
    if (isValid(transaction)) {
        saveToDatabase(transaction);
    }
}

private boolean isValid(Transaction t) {
    return t.getAmount() < t.getSender().getBalance();
}

private void saveToDatabase(Transaction t) {
    String s = t.getSender() + "," + t.getRecipient() + "," + t.getAmount();
    database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
    // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
    // Read transactions from the database to determine the account balance
}
```

TEST VIA PUBLIC APIS

```
@Test
public void shouldTransferFunds() {
    processor.setAccountBalance("me", 150);
    processor.setAccountBalance("you", 20);

    processor.processTransaction(newTransaction()
        .setSender("me")
        .setRecipient("you")
        .setAmount(100));

    assertThat(processor.getAccountBalance("me")).isEqualTo(50);
    assertThat(processor.getAccountBalance("you")).isEqualTo(120);
}
```

- Such tests are more realistic and maintainable
- Less brittle, as internal refactoring won't affect the tests

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
    if (isValid(transaction)) {
        saveToDatabase(transaction);
    }
}

private boolean isValid(Transaction t) {
    return t.getAmount() < t.getSender().getBalance();
}

private void saveToDatabase(Transaction t) {
    String s = t.getSender() + "," + t.getRecipient() + "," + t.getAmount();
    database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
    // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
    // Read transactions from the database to determine the account balance
}
```



GOOD PRACTICE 1: TEST VIA PUBLIC APIS

- Public APIs change much less frequently than internal implementations. Hence, tests on public APIs change less frequently (more maintainable)
- If tests work the same way as users, change that breaks a test may also break a user (i.e., test failures indicating real user-affecting problems)
- Bonus: such tests can serve as useful examples and documentation for users



GOOD PRACTICE 2: TEST BEHAVIORS, NOT METHODS

- Common patten: every production method has a corresponding @test method
- Convenient at first
- But as the method grow more complex, the test also grows in complexity and becomes less maintainable



TEST BEHAVIORS, NOT METHODS

```
public void displayTransactionResults(User user, Transaction transaction) {  
    ui.showMessage("You bought a " + transaction.getItemName());
```

The method to be tested

```
}
```

```
@Test
```

```
public void testDisplayTransactionResults() {  
    transactionProcessor.displayTransactionResults(  
        newUser  
  
        new Transaction("Some Item", dollars(3)));  
  
    assertThat(ui.getText()).contains("You bought a Some Item");  
}
```



TEST BEHAVIORS, NOT METHODS

```
public void displayTransactionResults(User user, Transaction transaction) {  
    ui.showMessage("You bought a " + transaction.getItemName());  
    if (user.getBalance() < LOW_BALANCE_THRESHOLD) {  
        ui.showMessage("Warning: your balance is low!");  
    }  
}
```

@Test

```
public void testDisplayTransactionResults() {  
    transactionProcessor.displayTransactionResults(  
        newUser  
  
        new Transaction("Some Item", dollars(3)));  
  
    assertThat(ui.getText()).contains("You bought a Some Item");  
}
```

The method to be tested **is updated**



TEST BEHAVIORS, NOT METHODS

```
public void displayTransactionResults(User user, Transaction transaction) {  
    ui.showMessage("You bought a " + transaction.getItemName());  
    if (user.getBalance() < LOW_BALANCE_THRESHOLD) {  
        ui.showMessage("Warning: your balance is low!");  
    }  
}
```

@Test

```
public void testDisplayTransactionResults() {  
    transactionProcessor.displayTransactionResults(  
        newUserWithBalance(  
            LOW_BALANCE_THRESHOLD.plus(dollars(2))),  
        new Transaction("Some Item", dollars(3)));  
  
    assertThat(ui.getText()).contains("You bought a Some Item");  
    assertThat(ui.getText()).contains("your balance is low");  
}
```



The method to be tested is updated

A method-driven test:
As the method under test becomes more complex and implements more functionality, its unit test will become increasingly convoluted and grow more and more difficult to work with.



TEST BEHAVIORS, NOT METHODS

```
public void displayTransactionResults(User user, Transaction transaction) {  
    ui.showMessage("You bought a " + transaction.getItemName());  
    if (user.getBalance() < LOW_BALANCE_THRESHOLD) {  
        ui.showMessage("Warning: your balance is low!");  
    }  
}
```

The method to be tested

Behavior 1

```
@Test  
public void displayTransactionResults_showsItemName() {  
    transactionProcessor.displayTransactionResults(  
        new User(), new Transaction("Some Item"));  
    assertThat(ui.getText()).contains("You bought a Some Item");  
}
```

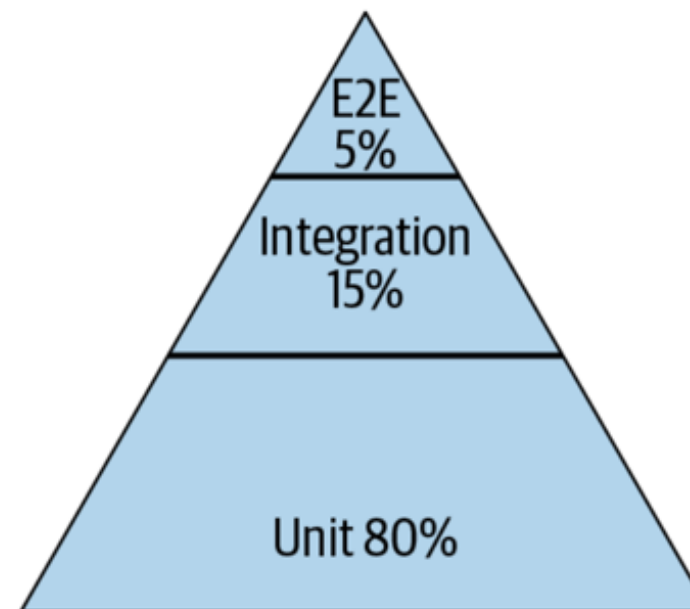
Behavior 2

```
@Test  
public void displayTransactionResults_showsLowBalanceWarning() {  
    transactionProcessor.displayTransactionResults(  
        newUserWithBalance(  
            LOW_BALANCE_THRESHOLD.plus(dollars(2))),  
        new Transaction("Some Item", dollars(3)));  
    assertThat(ui.getText()).contains("your balance is low");  
}
```

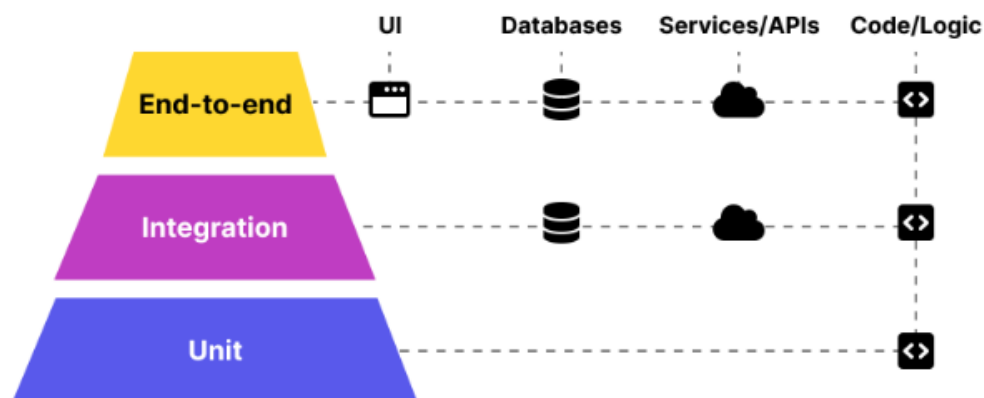
A behavior-driven test: rather than writing a test for each method, write a test for each behavior. ✓



TEST DOUBLES



WHY TEST DOUBLES?



- Imagine writing a test suite for a function that sends a request to an external server and then stores the response in a database.
- The test suite may
 - take hours to run
 - become flaky (不稳定) due to issues like random network failures or tests overwriting one another's data.
- Test doubles come in handy in such cases

WHAT ARE TEST DOUBLES?

A test double is an object or function that can stand in for a real implementation in a test, similar to how a stunt double can stand in for an actor in a movie.





TEST DOUBLES

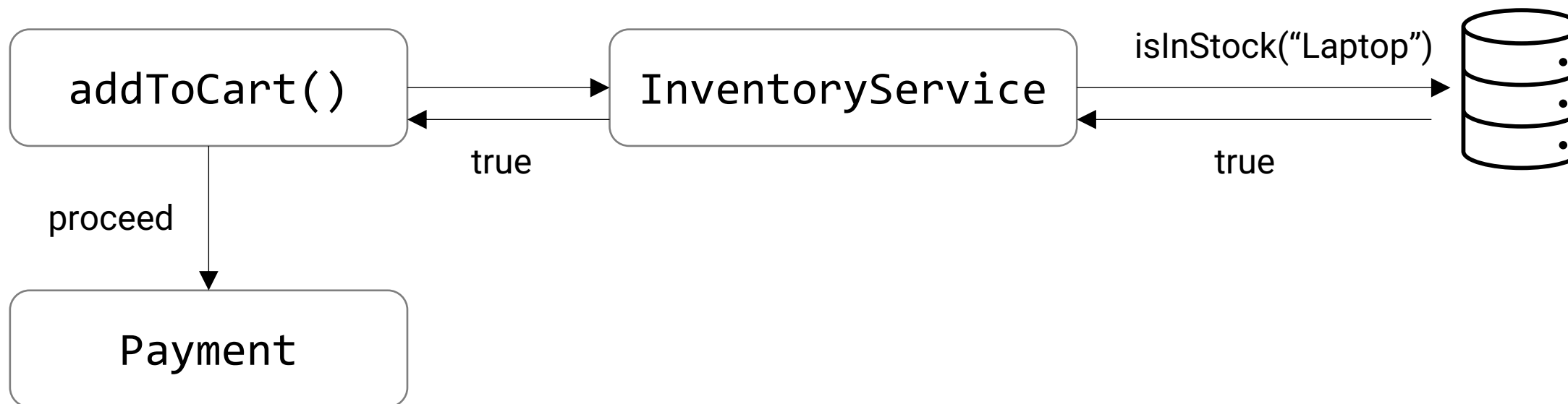
- Test doubles are replacement objects or code components that mimic the behavior of real objects or components in the SUT.
 - Test doubles are commonly used to isolate the SUT and its dependencies, such as external services or libraries, by replacing them with lightweight, controllable substitutes.
- Stubs
 - Fakes
 - Mocks



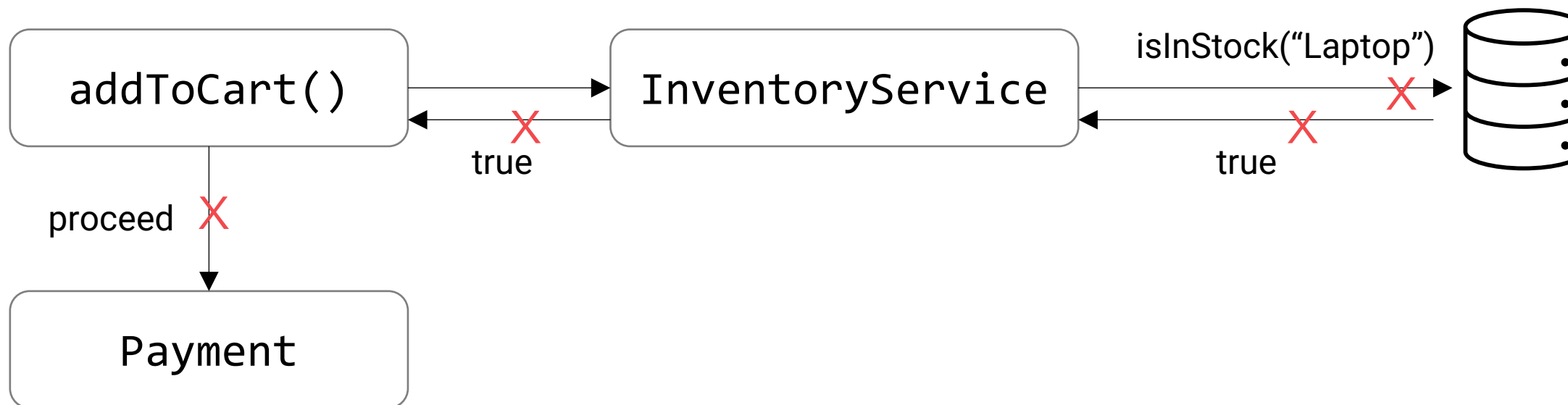
STUBS

- A stub returns predefined data to SUT
- A stub doesn't contain real logic; it's only there to respond to specific inputs with specific outputs.
- Stub is used when we cannot or don't want to involve objects that would answer with real data or have undesirable side effects.

SAMPLE WORKFLOW

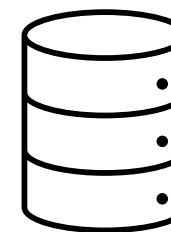
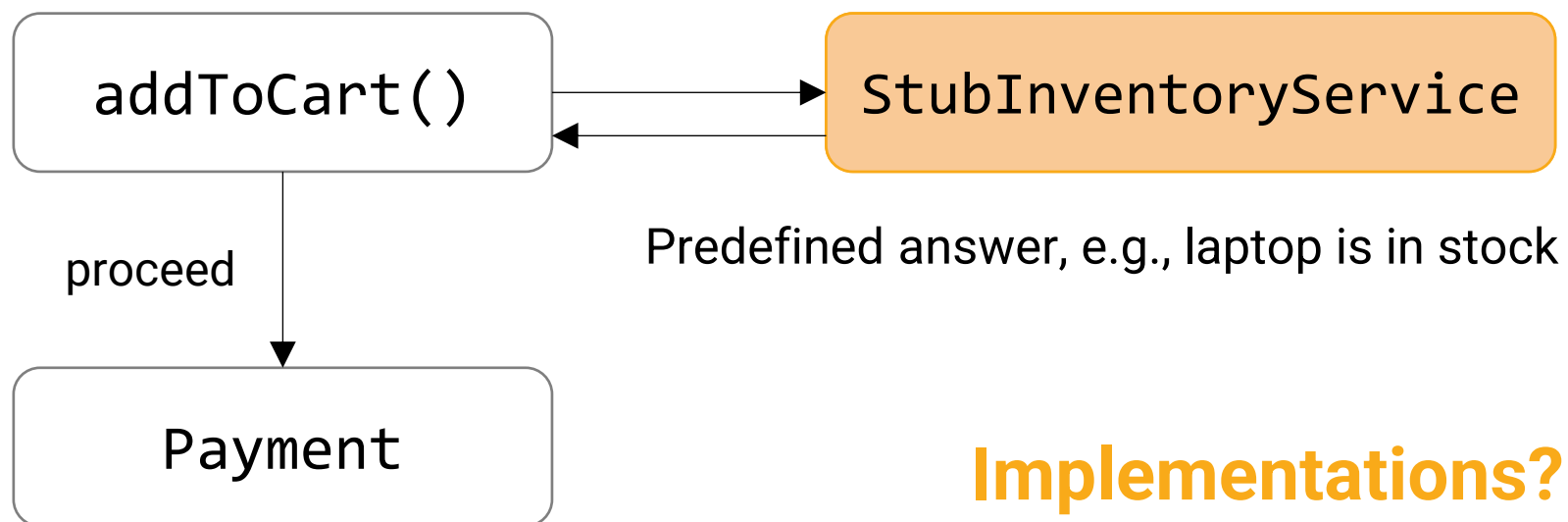


SAMPLE WORKFLOW





SAMPLE WORKFLOW



Implementations?



SAMPLE CODE

```
class ShoppingCart {  
    private final InventoryService inventoryService;  
  
    ShoppingCart(InventoryService inventoryService) {  
        this.inventoryService = inventoryService;  
    }  
  
    boolean addToCart(String itemName) {  
        if (inventoryService.isInStock(itemName)) {  
            return true;  
        }  
        return false;  
    }  
}
```

```
interface InventoryService {  
    boolean isInStock(String itemName);  
}
```

```
// Stub implementation  
class InventoryServiceStub implements InventoryService {  
    @Override  
    public boolean isInStock(String itemName) {  
        // Predefined responses for testing  
        if ("Laptop".equals(itemName)) return true;  
        if ("Headphones".equals(itemName)) return false;  
        return true; // default  
    }  
}
```



SAMPLE CODE

```
@Test
void unitTest_addToCartOnly() {
    InventoryService stub = new InventoryServiceStub();
    ShoppingCart cart = new ShoppingCart(stub);

    assertTrue(cart.addToCart("Laptop"));
    assertFalse(cart.addToCart("Headphones"));
}
```

InventoryServiceStub can be easily swapped to real InventoryService

```
@Test
void integrationTest_addToCartThenPaymentWorkflow() {
    InventoryService stub = new InventoryServiceStub();
    ShoppingCart cart = new ShoppingCart(stub);

    assertTrue(cart.addToCart("Laptop"));
    assertTrue(cart.payment()); // successful payment

    assertFalse(cart.addToCart("Headphones"));
    assertFalse(cart.payment()); // payment failed since cart is empty
}
```



FAKES

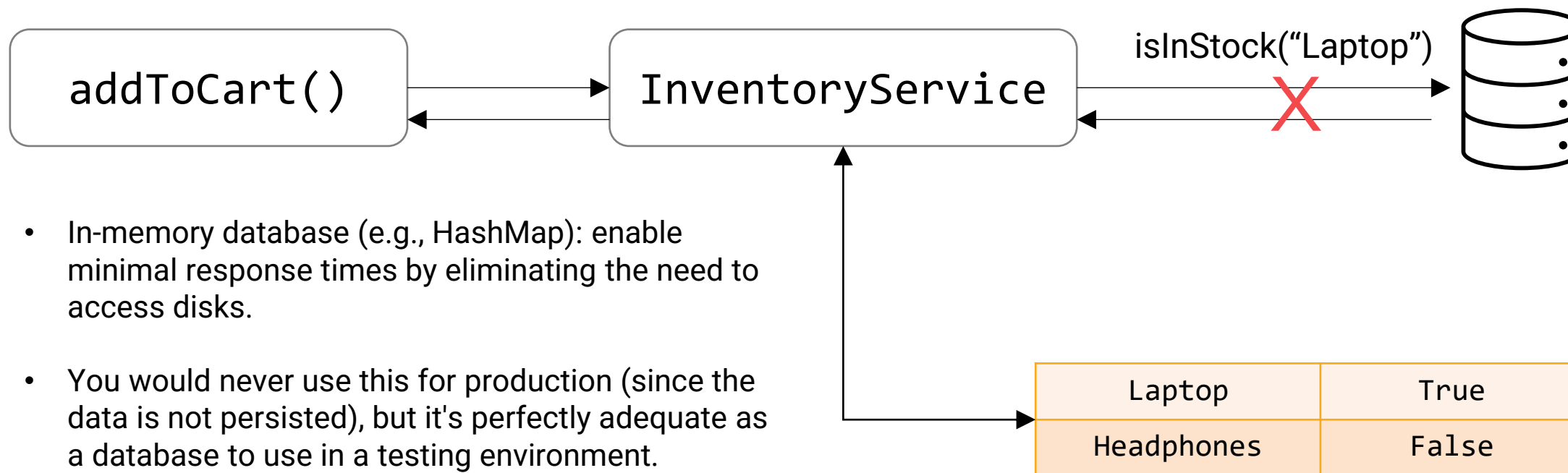
- A fake is a **working implementation** of a dependency, but **simplified** compared to the real one
- Unlike stubs, fakes may contain **actual logic** (not just predefined answers)
- Typically used to simulate external systems like DB in a lightweight way
- Useful for early development or faster integration tests



SAMPLE WORKFLOW



SAMPLE WORKFLOW





SAMPLE CODE

```
// Real database class
class InventoryDatabase {
    public boolean isItemInStock(String item) {
        // Connects to real database and checks stock
        return false; // placeholder
    }
}

class FakeInventoryDatabase extends InventoryDatabase {
    private Map<String, Boolean> stock = new HashMap<>();

    public FakeInventoryDatabase() {
        stock.put("Laptop", true);
        stock.put("Headphones", false);
    }

    @Override
    public boolean isItemInStock(String item) {
        return stock.getDefault(item, false);
    }
}
```



SAMPLE CODE

Easy swap: fake for tests, real DB in production

```
@Test
void testAddToCartWithFakeDB() {
    InventoryDatabase db = new FakeInventoryDatabase();

    InventoryService inventoryService = new InventoryService(db);
    ShoppingCart cart = new ShoppingCart(inventoryService);

    assertTrue(cart.addToCart("Laptop"));
    assertFalse(cart.addToCart("Headphones"));
    assertEquals(1, cart.getItems().size());
    assertEquals("Laptop", cart.getItems().get(0));
}
```

```
// Real database class
class InventoryDatabase {
    public boolean isItemInStock(String item) {
        // Connects to real database and checks stock
        return false; // placeholder
    }
}
```

```
class FakeInventoryDatabase extends InventoryDatabase {
    private Map<String, Boolean> stock = new HashMap<>();

    public FakeInventoryDatabase() {
        stock.put("Laptop", true);
        stock.put("Headphones", false);
    }

    @Override
    public boolean isItemInStock(String item) {
        return stock.getDefault(item, false);
    }
}
```

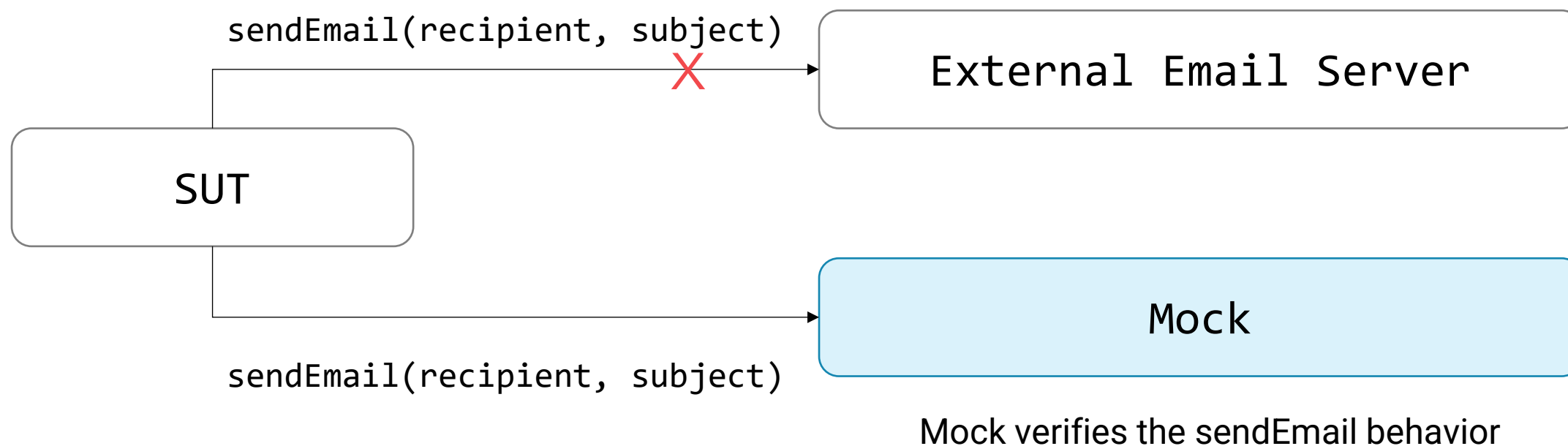


MOCKS

- A mock focuses on **behavior verification**
- Purpose: make assertions about how your SUT **interacted** with the dependency (whether a function in SUT is called in the correct way).
- Test fails if function
 - Not called
 - Called too many times
 - Called with wrong arguments
- We use mocks when we don't want to invoke production code or when there is no easy way to verify that intended code was executed.

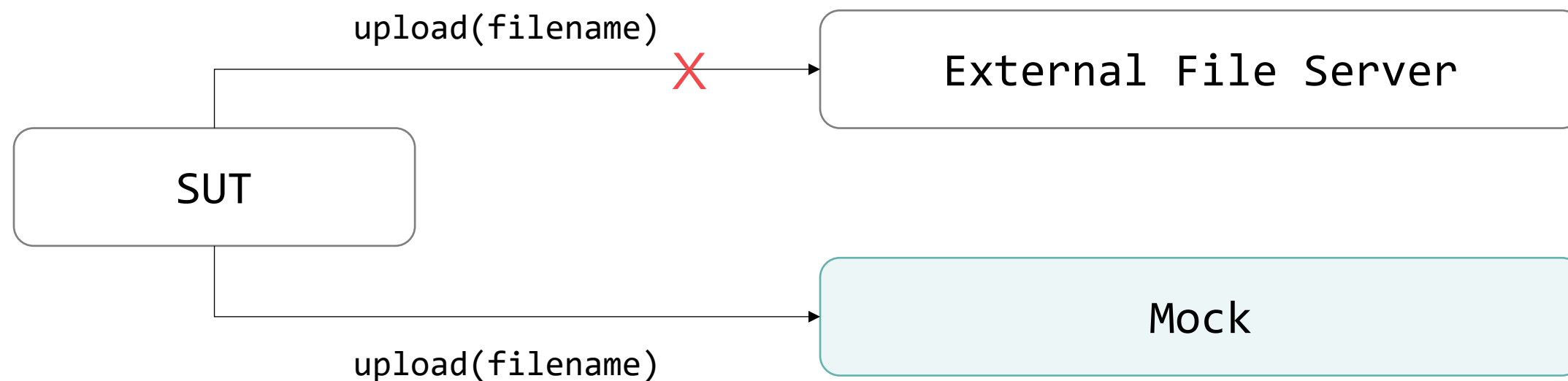


EXAMPLE 1





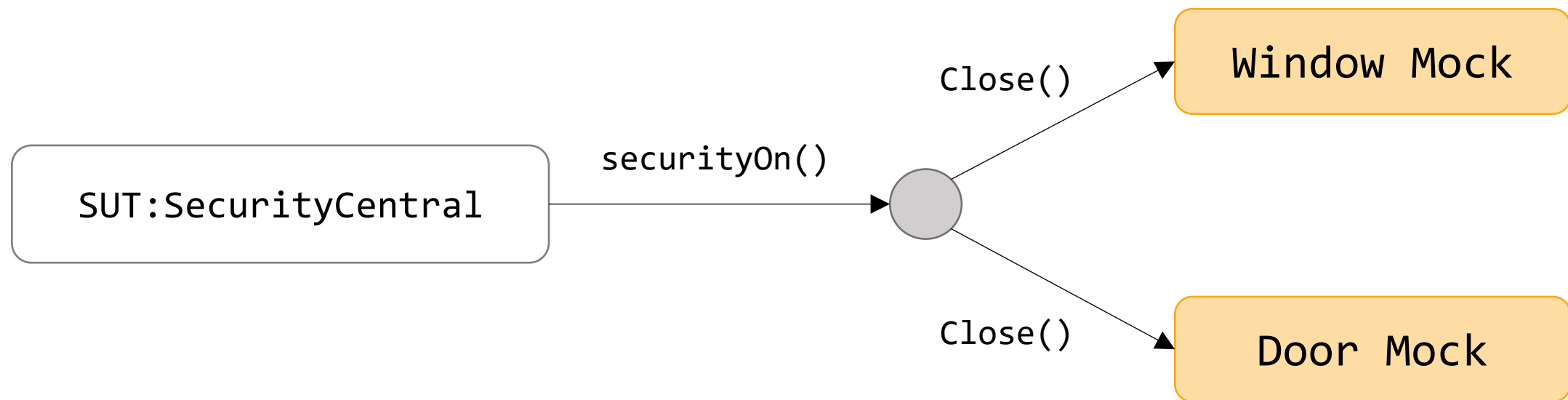
EXAMPLE 2



Mock verifies the file upload behavior



EXAMPLE 3



Mock verifies the close behavior



SAMPLE CODE

- Real payment service connects to credit cards: not suitable for unit tests
- SUT: PaymentProcessor
- Dependency: CreditCardService
- Test Target: chargeCreditCard

Example 13-1. A credit card service

```
class PaymentProcessor {  
    private CreditCardService creditCardService;  
    ...  
    boolean makePayment(CreditCard creditCard, Money amount) {  
        if (creditCard.isExpired()) { return false; }  
        boolean success =  
            creditCardService.chargeCreditCard(creditCard, amount);  
        return success;  
    }  
}
```

Code from the book "Software Engineering at Google"



SAMPLE CODE

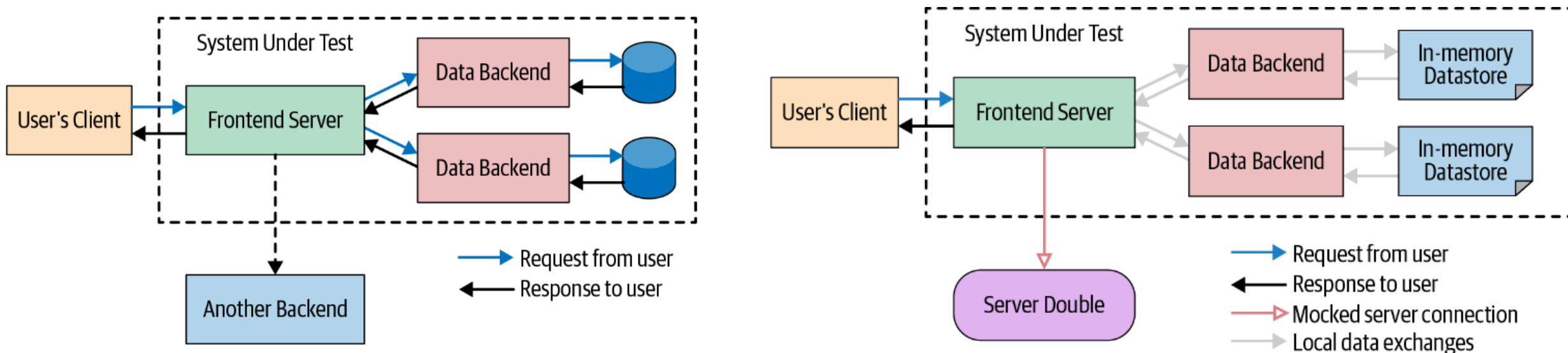
- Use Mockito to create a mock CreditCardService
- Mock lets us simulate success, failure, or edge cases
- Mocks also verify how dependencies are called
- Useful for testing critical interactions

Example 13-6. Mocking frameworks

```
class PaymentProcessorTest {  
    ...  
    PaymentProcessor paymentProcessor;  
  
    // Create a test double of CreditCardService with just one line of code.  
    @Mock CreditCardService mockCreditCardService;  
    @Before public void setUp() {  
        // Pass in the test double to the system under test.  
        paymentProcessor = new PaymentProcessor(mockCreditCardService);  
    }  
    @Test public void chargeCreditCardFails_returnFalse() {  
        // Give some behavior to the test double: it will return false  
        // anytime the chargeCreditCard() method is called. The usage of  
        // “any()” for the method’s arguments tells the test double to  
        // return false regardless of which arguments are passed.  
        when(mockCreditCardService.chargeCreditCard(any(), any()))  
        .thenReturn(false);  
        boolean success = paymentProcessor.makePayment(CREDIT_CARD, AMOUNT);  
        assertThat(success).isFalse();  
    }  
}
```

Code from the book “Software Engineering at Google”

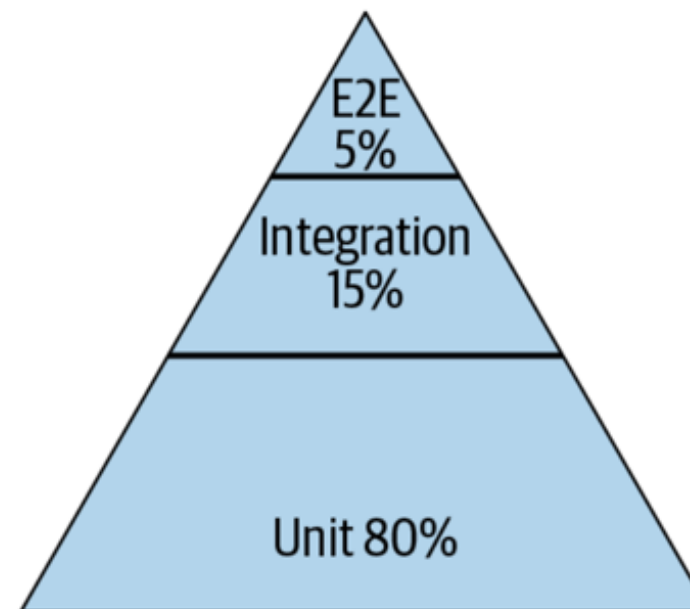
TEST DOUBLES SUMMARY



<https://abseil.io/resources/swe-book/html/ch14.html>



UI TESTING





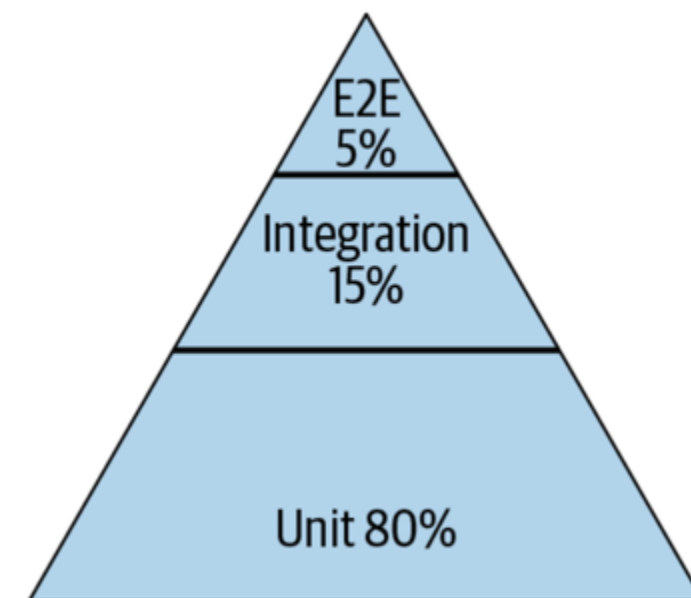
UI TESTING OVERVIEW

- UI is the visual part of a software that determines how users interact with the software and how information is displayed on the screen
- UI testing checks the application via the user interface, **in the same way an end user would**



UI TESTING OVERVIEW

- UI tests are very powerful because they go “end-to-end” from the UI layer down, which test all different parts of an application from beginning (UI) to end (the underlying service, database, etc.)
- UI testing can
 - Determine whether an application satisfies requirements
 - Demonstrate that it is fit for purpose
 - Detect defects





WHAT TO LOOK FOR IN UI TESTING?

- **Consistency:** consistency of colors, font types, and other visual elements.
- **Typography:** Is the text easily readable? Does it have enough contrast with the background?



✖ Not so good

This is an
amazing title

This section is badly designed and
you won't be able to differentiate
what's most important in it.

✔ Good

This is an
amazing title

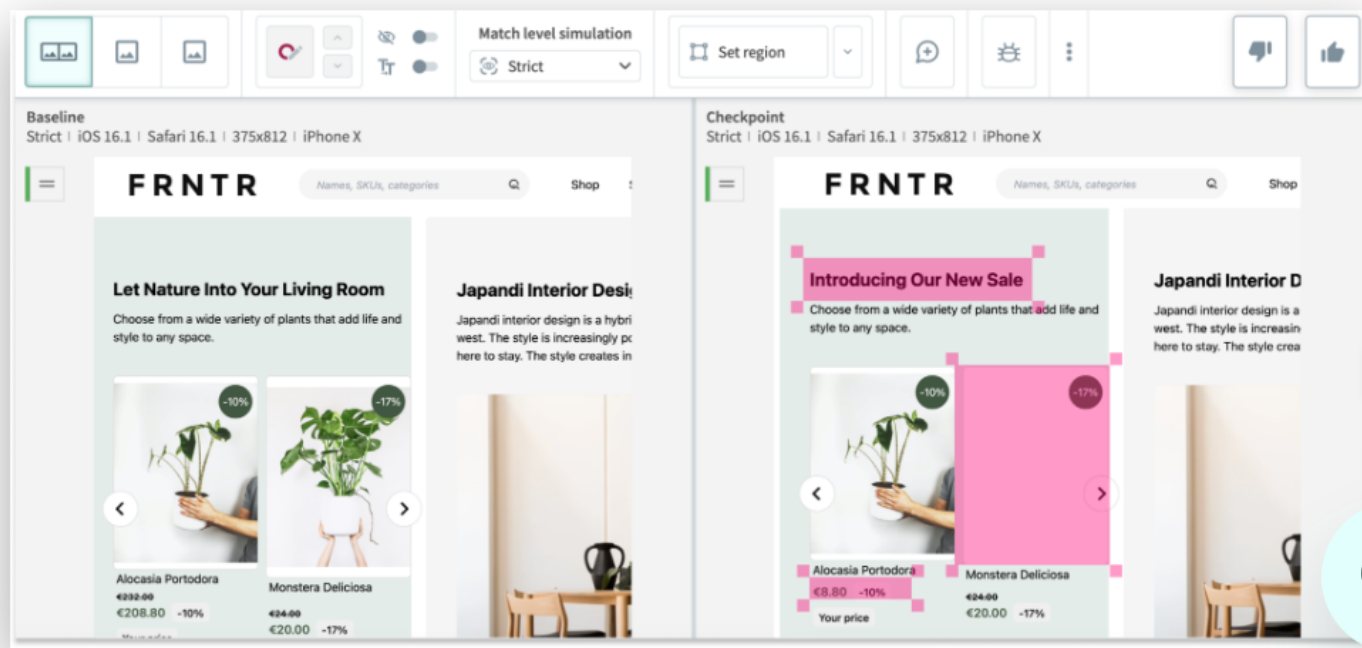
This section has a good design and
you can easily differentiate what is
title/description, and you can
establish what's more important

Mostly manual



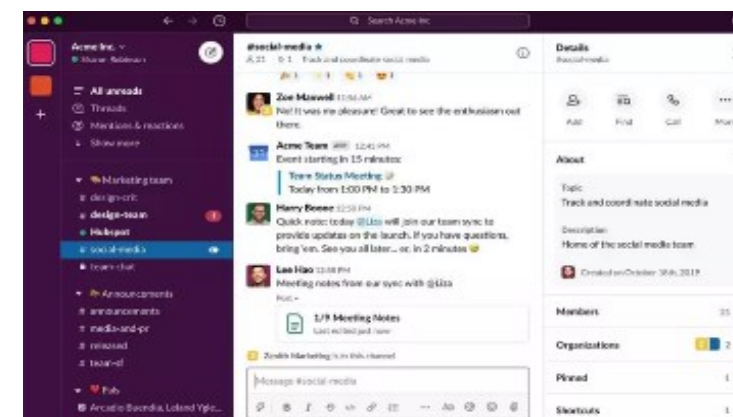
Test websites, apps, or docs with speed and precision using Visual AI.

Applitools Eyes uses AI to help teams quickly ship high-quality applications on any browser or device by replicating the 'human eye' and automatically spotting bugs and defects with every release.



WHAT TO LOOK FOR IN UI TESTING?

- **Behavior of interactive elements:** Do the buttons, clicks, hovers, drag-and-drops, etc. work as intended?
- **Functional validation:** Do inputs produce the expected outputs?
- **Adaptability:** Ensure that UI elements adapt and are displayed correctly in different devices, browsers, screen formats and sizes, and OS.



Can be automated

UI TESTING TOOLS

Example of UI functional validation:

- If you “add an item to cart”, you need to validate that the correct item, price, and quantity are properly added to the cart.
- Is there validation in place to prevent users from entering unreasonable data such as negative numbers or special characters that aren’t allowed?
- Can you verify that character length restrictions or required formats (e.g. phone number) aren’t violated?

UI testing tools (e.g., Selenium) allow developers to create programs that act like robots, **automating browser actions** to check for any issues or bugs automatically.



agiletravel-selenium-webdriver.tpr - [C:\work\agiletravel-ui-tests\selenium-webdriver-rspec\spec\flight_spec.rb] - TestWise 6.6.5

File Edit Search View Navigate Script Refactor Run Tools Window Help Develop

Env: production

PROJECT EXPLORER

- agiletravel-selenium-webdriver
 - pages
 - abstract_page.rb
 - flight_page.rb
 - login_page.rb
 - passenger_page.rb
 - payment_page.rb
 - spec
 - debugging_spec.rb
 - demo_ajax_spec.rb
 - flight_spec.rb
 - login_spec.rb
 - passenger_spec.rb
 - payment_spec.rb
 - spec_helper.rb
 - Rakefile
 - agiletravel-selenium-webdriver.tpr
 - agiletravel-selenium-webdriver.tws
 - agileway_utils.rb
 - buildwise.rake
 - buildwise_rspec_formatter.rb
 - install-lib.cmd
 - test_helper.rb

flight_spec.rb

```

22  driver.quit unless debugging?
23  end
24
25  it "[3] Return trip" do
26    flight_page = FlightPage.new(driver)
27    flight_page.select_trip_type("return")
28    flight_page.select_depart_from("Sydney")
29    flight_page.select_arrive_at("New York")
30
31    flight_page.select_depart_day("02")
32    flight_page.select_depart_month("May 2016")
33    flight_page.select_return_day("04")
34    flight_page.select_return_month("June 2016")
35    flight_page.click_continue
36
37    expect(page_text).to include("2016-05-02 Sydney to New York")
38    expect(page_text).to include("2016-06-04 New York to Sydney")
39  end
40
41  it "[2] One-way trip" do
42    flight_page = FlightPage.new(driver)
43    flight_page.select_trip_type("oneway")
44    flight_page.select_trip_type("return")
45    flight_page.select_depart_from("Sydney")
46    flight_page.select_arrive_at("New York")
47
48    flight_page.select_depart_day("02")
49    flight_page.select_depart_month("May 2016")
50    flight_page.click_continue
51
52    expect(page_text).to include("2016-05-02 Sydney to New York")
53  end
54
55
56  end
57

```

Line: 35 Col: 31 RSpec production: <https://travel.agileway.net>

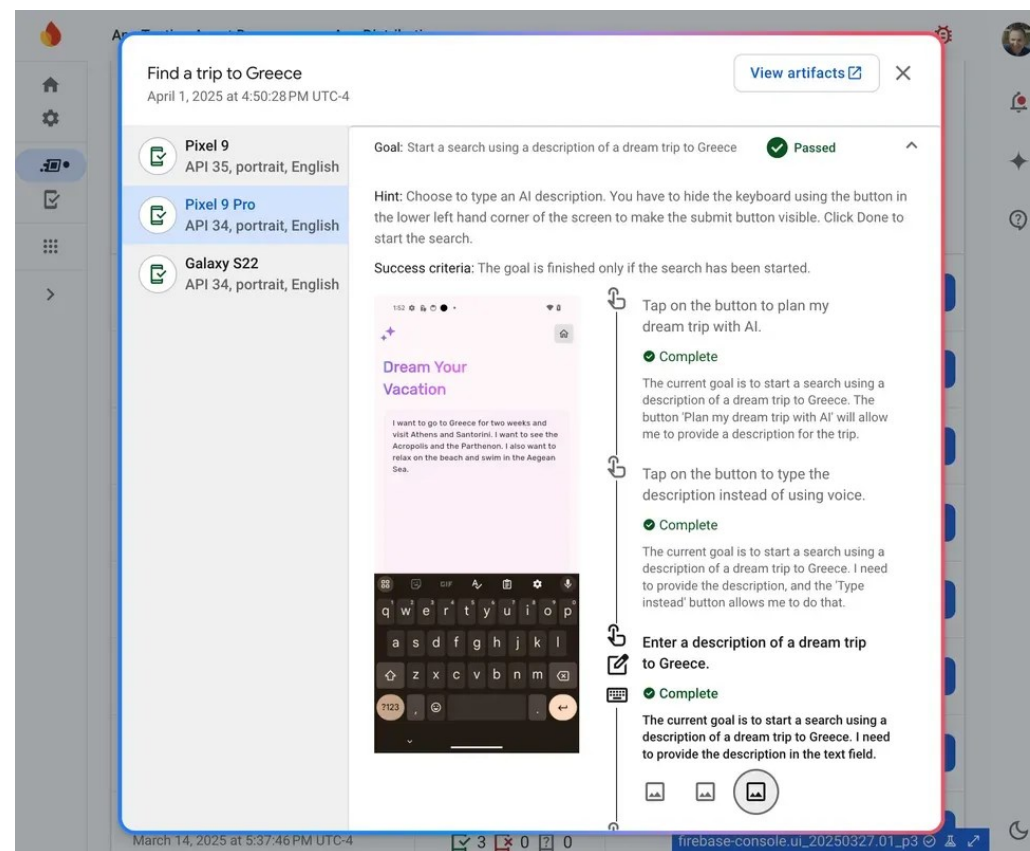
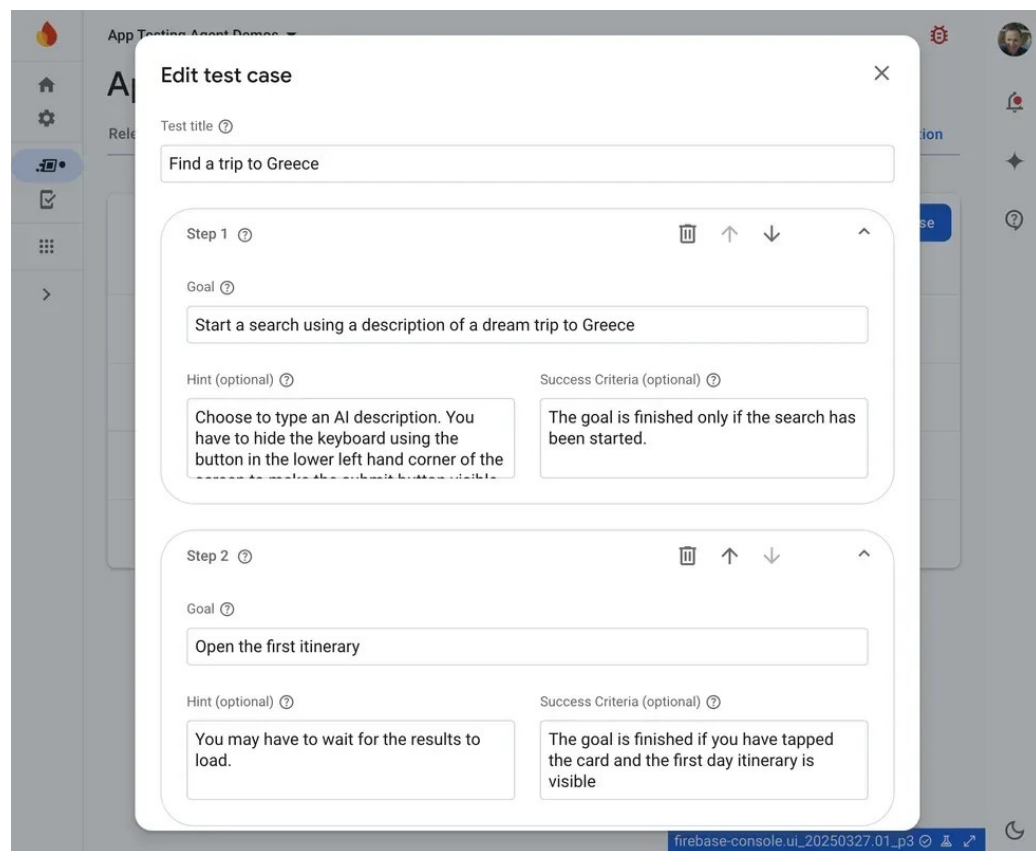


Selenium

<https://medium.com/geekculture/headless-browser-testing-clarified-d544ef0acf43>



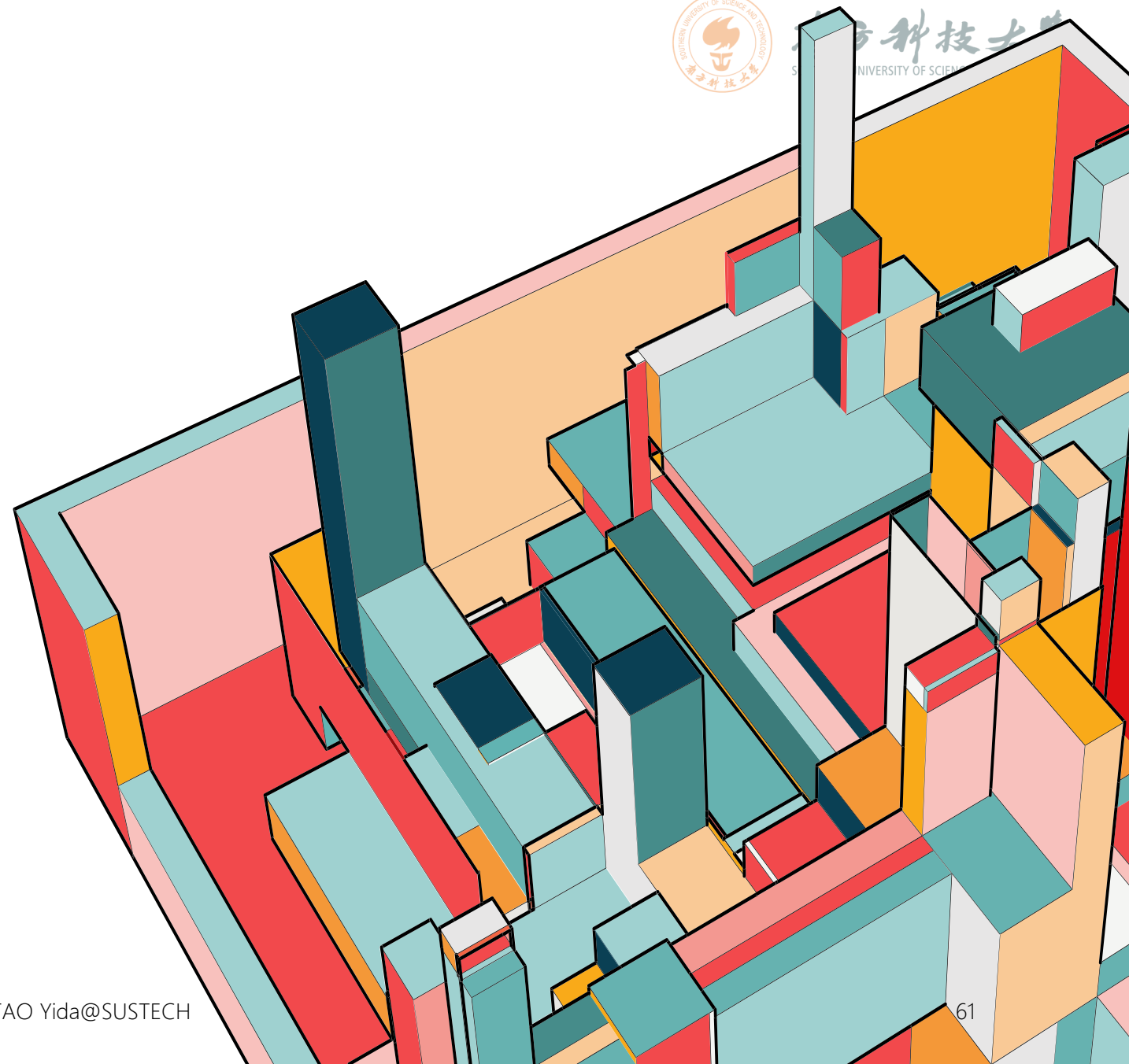
AI-POWERED TESTING AGENT





READINGS

- Chapter 11-13. Software Engineering at Google by Winters et al
- Chapter 21. Software Testing: A Craftsman's Approach. Paul C. Jorgensen.





NEXT

- CI/CD